

Local Variables and Bindings

CS 350

Dr. Joseph Eremondi

May 7, 2025

Overview: Variables

Overview: Variables

Syntax for Variables

- Want a way to:

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
  ...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
  ...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program
- New syntax:

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program
- New syntax:
 - Variables are expressions

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program
- New syntax:
 - Variables are expressions
 - Anywhere you can use a number, you can use a variable representing a number

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program
- New syntax:
 - Variables are expressions
 - Anywhere you can use a number, you can use a variable representing a number
 - `let1` for defining variables

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Syntax for Variables

- Want a way to:
 - Define variables as having certain values
 - Use those values later in a Curly program
- New syntax:
 - Variables are expressions
 - Anywhere you can use a number, you can use a variable representing a number
 - `let1` for defining variables
 - Called `let1` because we can only define 1 variable at a time

```
<expr> ::=  
| <number>  
| <boolean>  
| <var> ;; new  
...  
| | {let1 {<var> <expr>} <expr>}
```

Variable Binding

- `{let1 {x val} body}`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`
 - But with `x` replaced by value of `val`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`
 - But with `x` replaced by value of `val`
- We say that the `let1` expression **binds** the variable `x`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`
 - But with `x` replaced by value of `val`
- We say that the `let1` expression **binds** the variable `x`
 - `x` is **bound** in `body`

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`
 - But with `x` replaced by value of `val`
- We say that the `let1` expression **binds** the variable `x`
 - `x` is **bound** in `body`
 - e.g. it's defined in `body` and can be referred to there

Variable Binding

- `{let1 {x val} body}`
 - Can refer to `x` in `body`
 - Should give the value of `body`
 - But with `x` replaced by value of `val`
- We say that the `let1` expression **binds** the variable `x`
 - `x` is **bound** in `body`
 - e.g. it's defined in `body` and can be referred to there
 - Will shadow any previous definitions of `x` inside `body`

New Abstract Syntax

- Variable can be used as an expression

New Abstract Syntax

- Variable can be used as an expression
- `let t1` has the name of the variable, its value, and where it's used

New Abstract Syntax

- Variable can be used as an expression
- `let t1` has the name of the variable, its value, and where it's used

New Abstract Syntax

- Variable can be used as an expression
- `let1` has the name of the variable, its value, and where it's used

```
(define-type ExpS
  ...
  (varS [x : Symbol])
  (let1S [var : Symbol]
        [val : ExpS]
        [body : ExpS]))
```

```
(define-type Exp
  ...
  (varE [x : Symbol])
  (let1E [var : Symbol]
        [val : Exp]
        [body : Exp]))
```

- Nothing different, just recursively desugar the parts

- Nothing different, just recursively desugar the parts

Desugaring

- Nothing different, just recursively desugar the parts

```
;; Desugar Expressions into Core Syntax
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    ...
    [(varS x)
     (varE x)]
    [(let1S var val body)
     (let1E var (desugar val) (desugar body))]
    ))
```

What We Need to Interpret

What We Need to Interpret

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(let1E var val body)
     (let* ([valV (interp val)]
            [subbedBody TODO])
      (interp subbedBody))])
```

- Evaluate the value for the variable `var`

What We Need to Interpret

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(let1E var val body)
     (let* ([valV (interp val)]
            [subbedBody TODO])
      (interp subbedBody))])
```

- Evaluate the value for the variable `var`
- Replace `var` with that value in `body`

What We Need to Interpret

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(let1E var val body)
     (let* ([valV (interp val)]
            [subbedBody TODO])
      (interp subbedBody))])
```

- Evaluate the value for the variable `var`
- Replace `var` with that value in `body`
 - Need a new helper function to do this!

What We Need to Interpret

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(let1E var val body)
     (let* ([valV (interp val)]
              [subbedBody TODO])
              (interp subbedBody))])
```

- Evaluate the value for the variable `var`
- Replace `var` with that value in `body`
 - Need a new helper function to do this!
- Then, evaluate the body *after the substitution*

Substitution

How can we replace a variable in an expression?

How can we replace a variable in an expression?

- More recursion!

How can we replace a variable in an expression?

- More recursion!
- “Replace all occurrences of the variable x with the expression s inside of the expression t ”

How can we replace a variable in an expression?

- More recursion!
- “Replace all occurrences of the variable x with the expression s inside of the expression t ”
- Do this by traversing the expression recursively

How can we replace a variable in an expression?

- More recursion!
- “Replace all occurrences of the variable x with the expression s inside of the expression t ”
- Do this by traversing the expression recursively
- Critical operation in programming languages

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable
- $\text{let } t_1$ expressions: replace in the body and the value

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable
- $\text{let } t_1$ expressions: replace in the body and the value
 - **unless** the bound variable is the variable we're replacing

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable
- $\text{let } t_1$ expressions: replace in the body and the value
 - **unless** the bound variable is the variable we're replacing
 - Then, only substitute in the `value`

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable
- `let` expressions: replace in the body and the value
 - **unless** the bound variable is the variable we're replacing
 - Then, only substitute in the `value`
 - since the variable in the body is a “new” shadowing variable

Cases for Substitution

- Variable y : check if $y = x$ i.e. it's the variable the one we're replacing
 - If it is, produce s
 - Otherwise, produce y again
- Literal: just return the literal unchanged
 - since it doesn't contain the variable
- `let` expressions: replace in the body and the value
 - **unless** the bound variable is the variable we're replacing
 - Then, only substitute in the value
 - since the variable in the body is a “new” shadowing variable
- Everything else: recursively substitute in the sub-expressions

Code for Substitution: Base Cases

- Literals: don't do anything

Code for Substitution: Base Cases

- Literals: don't do anything
- Variables: replace if it's what we're trying to replace

Code for Substitution: Base Cases

- Literals: don't do anything
- Variables: replace if it's what we're trying to replace
 - Otherwise, don't do anything

Code for Substitution: Base Cases

- Literals: don't do anything
- Variables: replace if it's what we're trying to replace
 - Otherwise, don't do anything

Code for Substitution: Base Cases

- Literals: don't do anything
- Variables: replace if it's what we're trying to replace
 - Otherwise, don't do anything

```
(define (subst [toReplace : Symbol]
               [replacedBy : Exp]
               [e : Exp] ) : Exp
  (type-case Exp e
    [(varE x)
     (if (symbol=? x toReplace)
         replacedBy
         (varE x))]
    [(numE n)
     (numE n)]
    [(boolE b)
     (boolE b)]
```

Code for Substitution: Bound Variable

- Replace variable in value and body *unless* that variable is bound in body

Code for Substitution: Bound Variable

- Replace variable in value and body *unless* that variable is bound in body

Code for Substitution: Bound Variable

- Replace variable in value and body *unless* that variable is bound in body

```
(define (subst [toReplace : Symbol]
               [replacedBy : Exp]
               [e : Exp])
  : Exp
  (type-case Exp e
    ...
    [(let1E var val body)
     (let1E var
       (subst toReplace replacedBy val)
       (if (symbol=? var toReplace)
           body
           (subst toReplace replacedBy body)))]])
```

Code for Substitution: Recursive Cases

- Just recursively substitute in the parts

Code for Substitution: Recursive Cases

- Just recursively substitute in the parts

Code for Substitution: Recursive Cases

- Just recursively substitute in the parts

```
(define (subst [toReplace : Symbol]
               [replacedBy : Exp]
               [e : Exp])
  : Exp
  (type-case Exp e
    ...
    [(plusE l r)
     (plusE (subst toReplace replacedBy l)
              (subst toReplace replacedBy r))]
    [(timesE l r)
     (timesE (subst toReplace replacedBy l)
               (subst toReplace replacedBy r))]
    [(cndE test thn els)
     (cndE (subst toReplace replacedBy test)
            (subst toReplace replacedBy thn)
            (subst toReplace replacedBy els))]
    [(zero?E e)
     (zero?E (subst toReplace replacedBy e))])
```

Lifting Values back into Expressions

- Substitution works on expressions, not values

Lifting Values back into Expressions

- Substitution works on expressions, not values
 - But want to plug values in for variables

Lifting Values back into Expressions

- Substitution works on expressions, not values
 - But want to plug values in for variables

Lifting Values back into Expressions

- Substitution works on expressions, not values
 - But want to plug values in for variables

```
(define (Value->Exp [v : Value])  
  : Exp  
  (type-case Value v  
    [(boolV b)  
     (boolE b)]  
    [(numV n)  
     (numE n)]))
```

Putting It All Together

- This gives us everything we need to interpret `let`

Putting It All Together

- This gives us everything we need to interpret `let`

Putting It All Together

- This gives us everything we need to interpret `let1`

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(let1E var val body)
     (let* ([valV (interp val)]
            [subbedBody (subst var (Value->Exp valV) body)]]
      (interp subbedBody))])
```

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let` 1

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let`
 - So it wasn't defined by `let`

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let1`
 - So it wasn't defined by `let1`
- It is an **undefined variable**

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let1`
 - So it wasn't defined by `let1`
- It is an **undefined variable**
 - Raise an error!

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let`
 - So it wasn't defined by `let`
- It is an **undefined variable**
 - Raise an error!
- Can detect this statically, but we won't because we're lazy

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let`
 - So it wasn't defined by `let`
- It is an **undefined variable**
 - Raise an error!
- Can detect this statically, but we won't because we're lazy

Interpreting Variables

- When interpreting `let`, we replace the bound `x` *before* we interpret the body
- What if we hit a variable when interpreting?
 - Then it wasn't replaced by interpreting `let1`
 - So it wasn't defined by `let1`
- It is an **undefined variable**
 - Raise an error!
- Can detect this statically, but we won't because we're lazy

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(varE x)
     (error 'interp
            (string-append "Undefined variable: "
                           (symbol->string x)))]
```